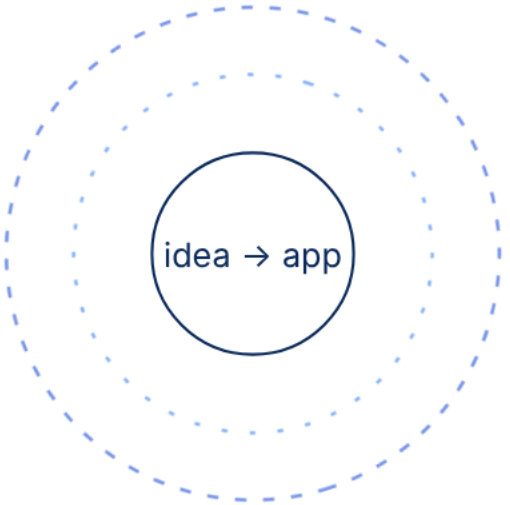


BUILDING PRODUCTS WITH CLAUDE · MODULE 2

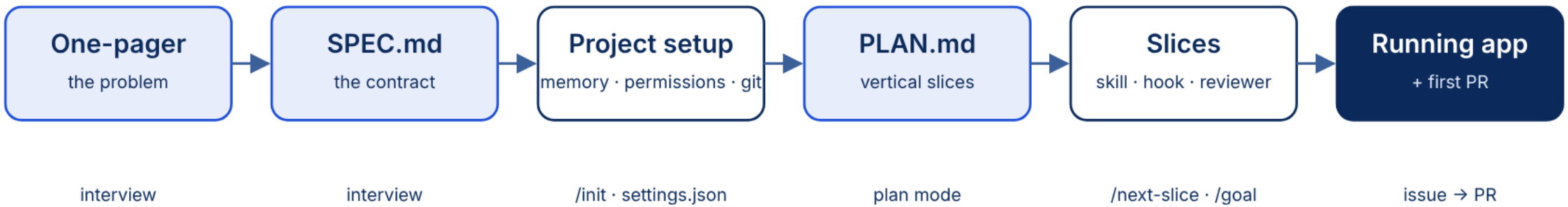
From Idea to a Running Application with Claude Code

Concepts, features and working practices for a development workflow in which the developer sets direction and reviews, and the model writes the code.

Manaranjan Pradhan
www.manaranjanp.com

OVERVIEW

The workflow produces six artefacts, in order



Each artefact is a file in the repository. The conversation is cleared between stages; the files carry the state.

CONCEPT

Division of work between the developer and the model

The developer

- Decides what to build and why
- Answers questions; resolves ambiguity
- Sets the rules: memory files, permissions
- Reviews every change before it is kept
- Approves irreversible actions (push, merge)

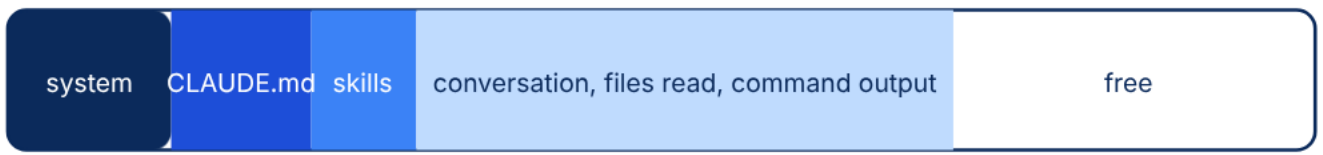
The model (Claude Code)

- Elicits requirements through structured questions
- Writes code, tests and documentation
- Runs commands and reads their output
- Follows the procedure in a skill; obeys hooks
- Reports what changed and what was verified

RULE The model does the typing. The developer remains the source of truth and the point of review.

CONCEPT

The context window is the resource to manage



Fixed cost on every turn: system prompt, memory files, skill and agent descriptions.

Variable cost: everything said, read and run in this session.

Full context → more money per turn, older instructions ignored, auto-compaction.

Tools: /context, /compact, /clear, the status line.

Three places state can live

Conversation — lost at `/clear`.
Files in the repository — SPEC, PLAN, code, tests: durable and reviewable.
Memory files — `CLAUDE.md`: loaded at the start of every session.

RULE Files carry the state, not the conversation. Clear the context when a stage ends.

PRACTICE

Memory and context management

Keep the fixed cost small

`CLAUDE.md` : only what the model cannot derive from the code; well under 200 lines. Personal notes in `CLAUDE.local.md` .

Long procedures belong in skills: a skill costs its description until it is invoked.

Unused MCP servers and plugins still load their tool names; turn them off. `/doctor` lists them with their cost.

A rule that must always hold is a hook, not a sentence with "IMPORTANT".

Keep the variable cost under control

`/clear` between unrelated tasks. After two failed corrections, clear and rewrite the prompt instead of correcting a third time.

`/compact` with a focus when a long task is not finished: `/compact keep the API changes and the test commands` .

Subagents keep large reads and noisy output out of the main context; ask them for short summaries back.

`/btw` for side questions; `/rewind` to undo file edits; `/usage` to see spend by skill, agent and server.

NOTE Compaction re-injects `CLAUDE.md` and auto memory from disk and re-reads up to five recently modified files. Everything else survives only if it is in a file.

FEATURE

Context and session commands

/context · /context all	What is in the context, by category, with the memory files that loaded and their token counts.
/compact [instructions]	Replace the history with a summary. The instructions say what the summary must keep.
/autocompact auto | <tokens>	Set the window at which automatic compaction runs: `/autocompact 500k` (100K–1M), `/autocompact auto` for the model default. Saved per model.
/clear [name]	Empty the context; the optional name labels the session being left so it can be resumed.
/doctor · /doctor prompt-audit	`/doctor` checks installation health and the context cost of skills, plugins, servers and hooks. `/doctor prompt-audit` audits `CLAUDE.md` and skills for outdated or conflicting instructions.
/advisor [model | off]	Sets a second, stronger model that the main model consults at decision points: before committing to an approach, when stuck on a recurring error, before declaring a task done. Experimental; its tokens count toward usage.
/model · /effort	Model and reasoning effort for the session.
/rename · /resume · claude --continue	Name the session; switch to another; reopen the most recent one in this directory.
/memory	List the memory files, toggle auto memory, open the memory folder.

FEATURE

Three kinds of input at the prompt

/command

Executed by Claude Code itself. Controls the session: `/clear`, `/context`, `/model`, `/effort`, `/compact`, `/usage`, `/statusline`, `/hooks`, `/reload-skills`. Also invokes skills by name: `/next-slice`.

Plain text

Sent to the model. Requests, answers to its questions, review comments. The model decides which tools to use.

!command

Executed by your shell. Output is shown and added to the context. Used today for Git: `!git status`, `!git add .`, `!git push`. On Windows this shell is Git Bash.

/model · /effort · /advisor	Choose the model, the reasoning effort, and optionally an advisor model consulted at decision points. Stronger settings for the specification and the plan; defaults for routine slices.
/btw <question>	Side question answered without being added to the main conversation.
/usage	Plan limits and spend for the session.
Shift+Tab	Cycles permission modes: default → accept edits → plan.

PRACTICE

Taking things out of the context

Every installed extension costs context whether or not it is used. Each kind has its own switch.

MCP server	Toggle it off in `/mcp` (written to `disabledMcpServers`): the server stays configured, its tools leave the context. Remove it with `claude mcp remove <name>`.
Plugin	`/plugin disable <name>` or `claude plugin disable <name>`: installed but inactive; none of its skills, agents, hooks or servers load. `claude plugin uninstall <name>` removes it.
Skill	A permission deny rule: `"deny": ["Skill(name)"]` in `settings.json` blocks that skill from being invoked. To remove it entirely, delete or move its folder.
Subagent	No per-name toggle. Remove or move its file out of `.claude/agents/` (project) or `~/.claude/agents/` (user).
Memory	`/memory` toggles auto memory; `CLAUDE.md` lines are removed by editing the file. Nested `CLAUDE.md` files and path-scoped rules load only when their paths are touched.
Check	`/context all` before and after; `/doctor` lists unused skills, plugins and servers with their context cost.

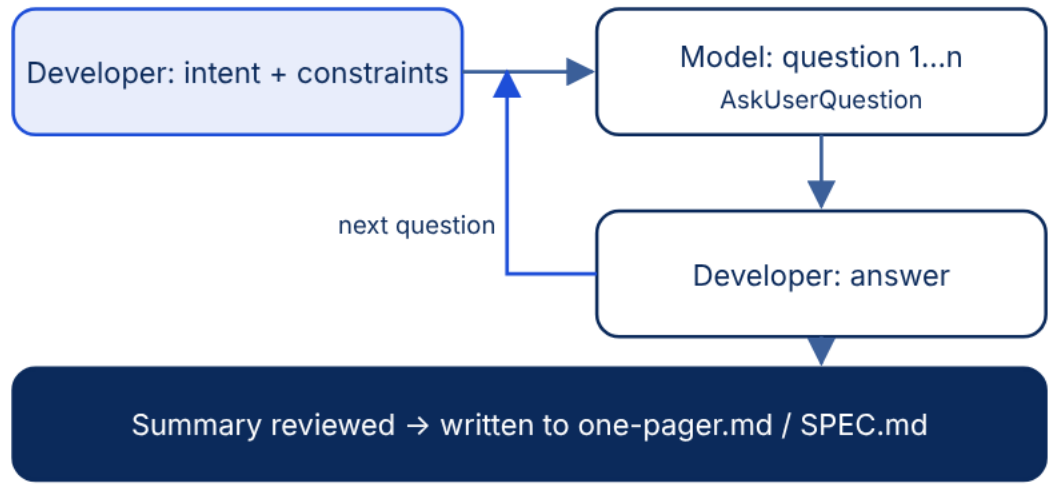
RULE Disable rather than uninstall when you may want it back: a disabled extension costs no context.

CONCEPT · ASKUSERQUESTION

Interview instead of dictate

The model has a tool, **AskUserQuestion**, that presents a structured question with options and waits for the answer. A prompt that asks for an interview turns the model into the elicitor and keeps the developer as the source of truth.

- The model asks about what it does not know; the developer answers from the business.
- Questions arrive one at a time; each answer shapes the next question.
- The summary at the end is reviewed before it is written to a file.



PRACTICE

Separate the problem interview from the solution interview

1 · One-pager (the problem)

Who the users are · what they ask · what they need to see · what must never happen · what is out of scope.
Excluded: technology, architecture, implementation.
4–5 questions · half a page

2 · SPEC.md (the solution)

Technical implementation · UI and UX · edge cases · concerns · trade-offs. "Do not ask obvious questions; dig into the hard parts."
Then: add the test assets; close every open item.
10–20 questions · the contract for all later work

WHY When technology enters the first conversation, the problem statement bends to fit it. The order protects the problem.

CONCEPT

Anatomy of a specification the model can build from

What and for whom

Users and roles · the question-to-answer pipeline · what the user sees (SQL, result, explanation) · conversation and follow-up behaviour · clarification and refusal behaviour.

Rules and formats

Safety rules (read-only, row cap, timeout) · configuration storage (`<ASKDB_CONFIG_DIR>/databases/<name>.yaml` , validated, atomic write) · the schema YAML and evaluation YAML formats, documented once.

Interfaces and checks

API · UI pages · error handling · testing strategy · the sample database and evaluation set as acceptance tests · **Open items**, emptied before planning.

PRACTICE Provided test assets (database, schema, evaluation set) are folded into the specification so that every later slice knows their shape.

CONCEPT · MEMORY

CLAUDE.md: standing instructions, loaded every session

Managed	Organisation-wide, set by administrators
~/.claude/CLAUDE.md	Personal, all projects
./CLAUDE.md	Project, committed, shared
./CLAUDE.local.md	Project, personal, not committed

All levels load at launch and are combined. A file created mid-session is read after `/clear` or a restart.

Belongs in CLAUDE.md

Stack and the exact commands to install, run, test, lint · folder layout · guardrails · working rules (source of truth, progress file, tests with every change) · commit conventions. **Under 60 lines.**

Does not belong

The specification · the plan · anything that changes weekly · long explanations. Each line costs tokens on every turn and dilutes the lines that matter.

CHECK `/context all` lists it under Memory files with its token count. Proof it is used: ask a question only that file can answer.

FEATURE · LOOKING INTO MEMORY

Where Claude Code keeps memory and transcripts

~/.claude/settings.json	User settings and permissions
~/.claude/history.jsonl	Prompt history
~/.claude/projects/<project>/memory/MEMORY.md	Auto memory index, one line per memory; loaded every session
~/.claude/projects/<project>/memory/<topic>.md	Memory details, read on demand
~/.claude/projects/<project>/<session-id>.jsonl	Full transcript: one JSON object per line (messages, tool calls, results)
… /subagents/ · /tool-results/	Beside each transcript
<repo>/CLAUDE.md · <repo>/.claude/	Project memory; skills, agents, hooks, settings

Auto memory	Plain Markdown. `cat` it, ask Claude to read it, or `/memory`. Only the first 200 lines / 25 KB of `MEMORY.md` load.
Transcripts	Plain, unencrypted JSON lines: `jq` or `cat` them. `claude --resume <path>` reopens that exact session. Swept after 30 days by default.
Readable export	`/export [file]` from inside a session writes the conversation as text.
One project	The project name comes from the Git repository, so worktrees and subfolders share one memory directory.

LAB Open `MEMORY.md` after the first slices: the "record learnings" rule in the skill writes there.

CONCEPT · PERMISSIONS

Allow the routine, ask for the irreversible, deny the secret

```
// .claude/settings.json (committed, shared)
{
  "permissions": {
    "allow": ["Edit", "Bash(pytest *)",
              "Bash(git add *)", "Bash(git commit *)"],
    "ask":   ["Bash(git push *)"],
    "deny":  ["Read(./.env)"]
  }
}
```

Rule syntax: `Tool` or `Tool(pattern)`. `Edit` also covers writes.

Evaluation order: deny, then ask, then allow; first match wins.

Files: `settings.json` is shared with the team; `settings.local.json` is personal and not committed. Managed settings override both.

Modes (Shift+Tab): default · acceptEdits · plan. `bypassPermissions` exists; it is not used in this course.

SECRETS The key lives in the environment. `.env` is ignored by Git and denied to the model; `.env.example` documents the variable names.

PRACTICE

Git from the first step, from inside the session

Commit before any code exists. The scaffold (CLAUDE.md, settings, docs, test assets) is the first commit; every later diff is small and readable.

Shell mode. `!git status` , `!git add .` , `!git commit -m "..."` run in your shell; output enters the context so the model sees the same state you do.

GitHub CLI. `gh auth login` once; both you and the model then use `gh` for issues and pull requests.

Push is an approval point. The `ask` rule stops the session until you confirm.

```
!git init
!git add .
!git commit -m "initial scaffold for the project"
!git remote add origin https://github.com/<you>/askdb.git
!git branch -M main
!git push -u origin main        # → ask: approve once
```

FEATURE

Plan mode: think before anything changes

In plan mode the model reads files, runs read-only commands and reasons, but does not edit files until the plan is approved. It is the mode for design decisions.

Enter	Shift+Tab until the status line reads *plan*, or `/plan`, or `claude --permission-mode plan`
Survives	`/clear`; check the status line
Leave	Shift+Tab back to default before asking for files to be written

Use it for

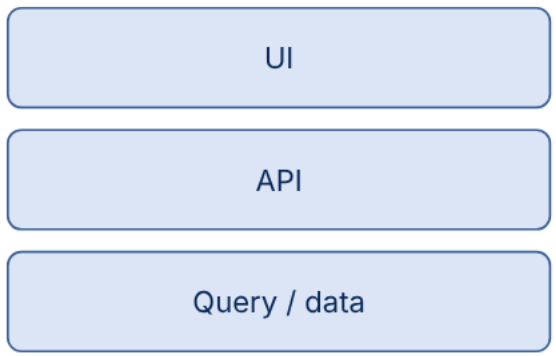
- Turning SPEC.md into an implementation plan
- Deciding how to approach a hard slice
- Exploring an unfamiliar codebase before changing it

The plan is produced in conversation; writing `docs/PLAN.md` is a separate, explicit request after leaving plan mode.

CONCEPT

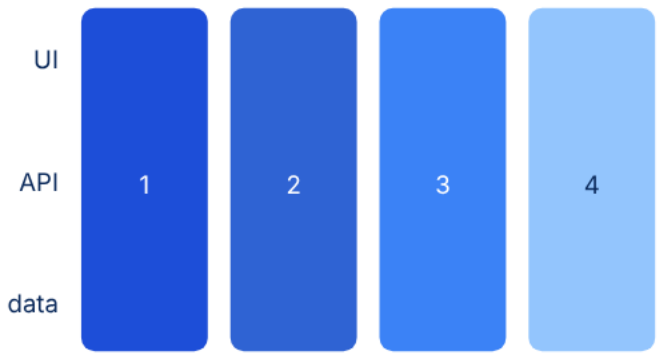
Thin vertical slices

Horizontal layers



Nothing works end to end until the last layer.
Integration problems appear at the end.

Vertical slices



Each slice crosses every layer for one capability.
Each is testable and shippable on its own.

A good slice

End to end · testable on its own · fits one short session · names its verification method and its dependency.
"SELECT-only executor with row cap, tested against cloudmetrics.db"

Not a slice

"Build the data layer"
A layer with nothing above it cannot be verified from the user's side.

PRACTICE

PLAN.md is read by tools, so its format is fixed

```
# docs/PLAN.md
- [x] **Slice 1: project skeleton**
  - Files: pyproject.toml, src/askdb/main.py, tests/test_health.py
  - Verify: `uv run pytest -q`; GET /health → 200
  - Depends on: —
- [ ] **Slice 2: configuration registry**
  - Files: src/askdb/registry.py, tests/test_registry.py
  - Verify: load, validate, atomic save; invalid YAML writes nothing
  - Depends on: 1
  - Note (from slice 1): settings are read from ASKDB_CONFIG_DIR
```

One checkbox line per slice, `- [ ] **Slice N: goal**`. The hook counts these lines; the skill finds the first unticked one.

Files, verification method and dependency under each slice. Nothing else.

Later runs leave one-line notes under the slice they concern.

Review the sequence before building: merge slices that are too small; split ones that are tightly coupled.

Commit the plan. It is the afternoon's work order.

CONCEPT

Three ways to extend the session

Skill

.claude/skills/<name>/SKILL.md

A procedure the model loads on demand.

Description read every turn;
body loaded when invoked.

Runs: the model, following steps.

Today: /next-slice

Hook

settings.json → hooks → event → command

A script that runs at a fixed event.

Deterministic: it always runs,
whatever the model decides.

Runs: your shell, as you.

Today: Stop → plan-status.sh

Subagent

.claude/agents/<name>.md

A separate model session with its own
context, prompt and tool list.
Returns a result to the main session.

Runs: a second Claude.

Today: slice-reviewer

Use a skill for a repeatable procedure, a hook for a rule that must never be skipped, a subagent for work that needs a clean context or different tools.

WALKTHROUGH · SKILL

A workflow skill: one task, explicit stop

```
---
name: next-slice
description: Implement the next unticked slice in docs/PLAN.md,
  verify it, commit, record learnings, and stop.
---
Work on exactly one slice.
1. Find the first unticked slice. State its number and goal.
2. Read every file it lists before changing anything.
3. Implement within scope; note out-of-scope needs under a later slice.
4. Run verification, then the full suite. Never weaken a test.
5. Too big for one session? Split it; finish the first half.
6. Record learnings: slice-specific → PLAN.md; project-wide → memory.
7. Tick the slice. Commit as slice N: <goal>.
8. Stop. Report: what changed, what was verified, what was recorded.
```

`name` becomes the `/next-slice` command.

`description` is in the context every turn; the body only when invoked. Keep the description precise so it triggers only when intended.

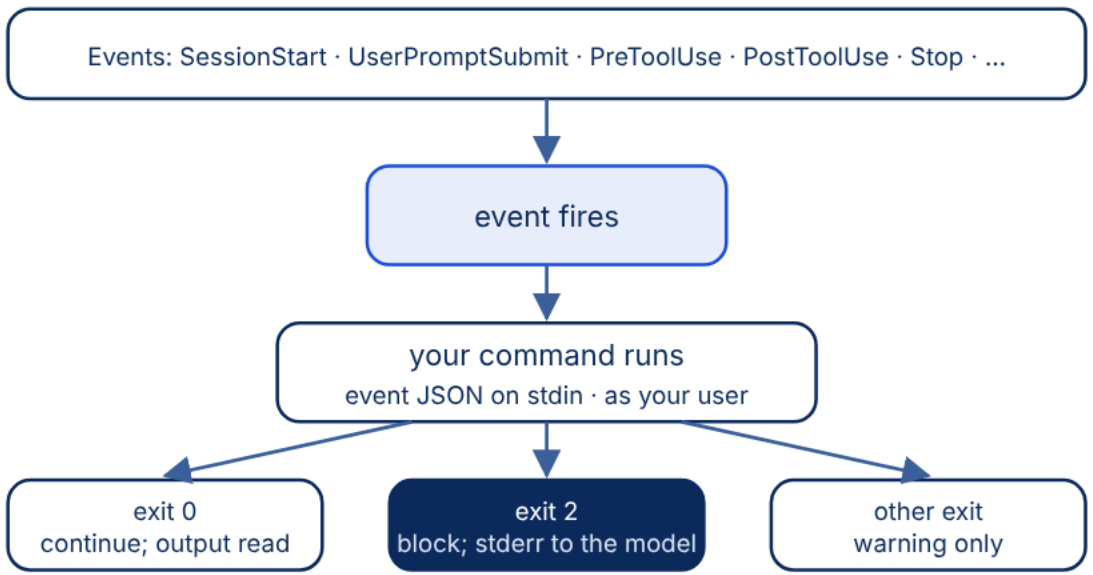
Rule 8 makes the loop reviewable: one slice, one commit, one review.

Other frontmatter: `allowed-tools`, `argument-hint`, `disable-model-invocation`.

`/reload-skills` after creating it; `/context all` shows its cost.

WALKTHROUGH · HOOK

A hook runs code at an event, every time




```
# .claude/hooks/plan-status.sh  (Stop event)
PLAN="$CLAUDE_PROJECT_DIR/docs/PLAN.md"
DONE=$(grep -c '^- \[x\] \*\*Slice' "$PLAN")
TOTAL=$(grep -c '^- \[[ x]\] \*\*Slice' "$PLAN")
jq -n --arg msg "PLAN.md: $DONE/$TOTAL slices done" \
    '{systemMessage: $msg}'
```

Registered in `settings.json` under `hooks → Stop`; `/hooks` lists it.

`{"systemMessage": ...}` shows a line to you after every response.

A `PreToolUse` hook with exit 2 blocks a tool call: the enforcement pattern used in the next module.

Windows: hooks run through Git Bash; keep `.sh` files with LF endings.

WALKTHROUGH · SUBAGENT

A reviewer that has not seen the work being done

```
---
name: slice-reviewer
description: Reviews the most recent slice commit against
  docs/SPEC.md and docs/PLAN.md. Read-only.
tools: Read, Grep, Glob, Bash(git diff *), Bash(git log *),
  Bash(git show *), Bash(uv run pytest *)
model: sonnet
---
1. git show HEAD: what changed.
2. Read the slice entry, the relevant SPEC sections, CLAUDE.md.
3. Check: scope · guardrails · tests would fail if the feature broke
   · anything that makes a later slice harder.
4. Run the suite.
5. Reply: VERDICT (APPROVE | REWORK), then ≤5 findings as
   file:line — problem — fix. No praise, no diff summary.
```

Own context. The reviewer starts clean; it is not grading its own work.

Own tools. Read-only plus Git inspection and the test runner. It cannot edit.

Own model. Cheaper or stronger as the job needs.

Fixed output contract. The main session can act on a verdict and a list; prose is harder to use.

Invoked by asking ("run the slice-reviewer"), by `@slice-reviewer`, or automatically when the description matches.

> COST No shared context: everything the reviewer needs must be in files or in the request.

PRACTICE

The implementation loop



One slice · one commit · one review. Review the tests as closely as the code: would they fail if the feature broke?

FEATURE

Running several slices unattended

`/goal` inside one session

`/goal complete slices 2–4 from docs/PLAN.md using /next-slice`

Claude Code keeps taking turns until the condition holds. Allowed commands run without prompts; a push stops and asks.

Limitation: the context is *not* cleared between slices; auto-compaction summarises it when full.

A shell loop with print mode

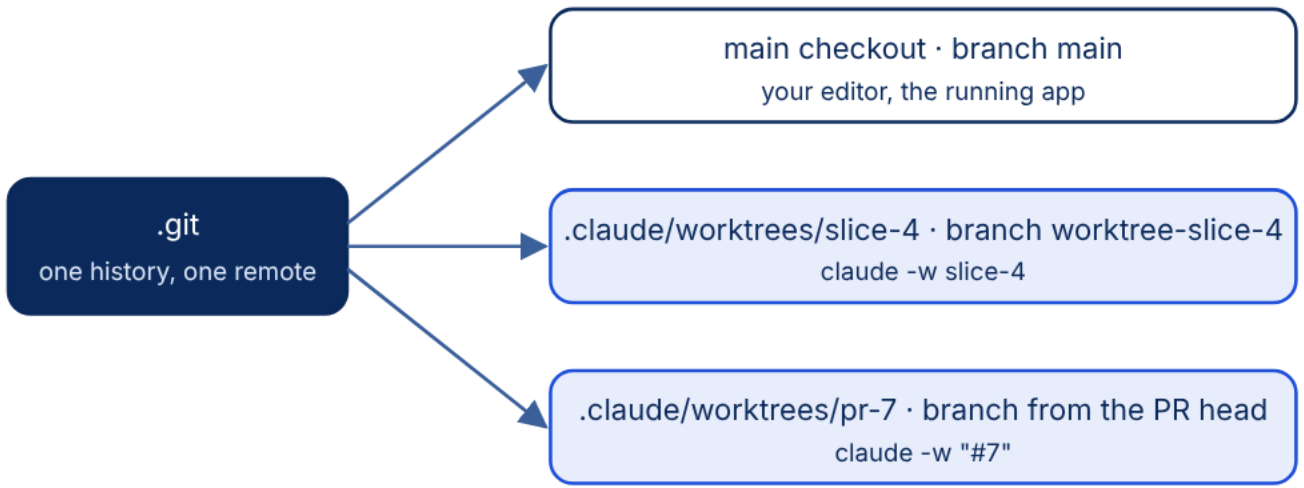
```
for i in 1 2 3; do
  claude -p "/next-slice" \
    --permission-mode acceptEdits || break
done
```

Each `claude -p` run is a new session: a fresh context per slice. The loop stops on the first failure. Review the commits afterwards.

RULE Unattended runs trade per-slice review for batch review. Acceptable for early, mechanical slices; not for the ones that touch the query pipeline or safety rules.

CONCEPT

Worktrees: an isolated checkout per task



A Git worktree is a second working directory on its own branch, sharing the repository's history.

`claude --worktree <name>` (or `-w`) starts a session in `.claude/worktrees/<name>`; edits to the main checkout are blocked while isolated.

Use it to run two slices in parallel, to trial a public plugin, or to review a pull request without disturbing your branch.

On exit, a clean unnamed worktree is removed; one with changes asks keep or remove. Add `.claude/worktrees/` to `.gitignore`.

Hooks: `$CLAUDE_PROJECT_DIR` stays at the project root; `cwd` follows the worktree.

FEATURE

Designing the UI and keeping the record

/design <brief>

Produces UI mockups, screen flows, landing pages or posters as artboards on one canvas, published as a Claude Design artifact on claude.ai. Open it in a browser to edit; edits save automatically; each artboard exports as PNG or PDF.
In the lab: `/design` from `PLAN.md` and `SPEC.md` → iterate in plain language → record the agreed pages and components in `PLAN.md` so the UI slices can find them.
Needs a claude.ai login with artifacts available; not on Bedrock, Vertex or Foundry.

/export [filename]

Writes the current conversation as readable text: messages and tool output rendered, not raw JSON. Without a filename it offers clipboard or file.
In the lab: export after each batch of slices (`/export slices_2_8.txt`) and keep it with the project for the retrospective.

/rename <name>	Name the session for later
claude --resume <name | id | path>	Reopen it, including from a transcript path
claude --continue	Reopen the most recent session here

PRACTICE

Reviewing code the model wrote

Read the diff

Scope: does the change do what the slice says, and nothing more? Out-of-scope edits belong in a note under a later slice, not in this commit.

Read the tests

Would this test fail if the feature broke? A test that asserts the code does what the code does proves nothing. Watch for weakened or skipped tests.

Check the guardrails

Read-only SQL, limits, no secrets in code or logs, configuration validated before it is written. The reviewer subagent checks these too; the human checks the reviewer.

SIGNAL A green suite after a model turn is a claim, not a proof. The claim is verified by reading, by the reviewer, and by running the application.

OPERATING THE APPLICATION

Configuration model and the runbook



Environment variables, not .env: ASKDB_CONFIG_DIR · ANTHROPIC_API_KEY · CLOUDMETRICS_DB_PATH (sample

Runbook written and verified by the model: every command in `docs/RUN_LOCAL.md` was executed before it was documented.

Two processes: API on 8000, web UI on 5173; variables set in the API terminal before start.

First account from the CLI; later accounts from the admin screen.

Acceptance: the sample questions with known answers; a refused write; a declined non-data question; the evaluation pass rate as the baseline for later modules.

PRACTICE

The unit of change: issue → branch → tests → pull request



The model works from the issue text, not from a chat description: the requirement is recorded where the team can see it.

/security-review diffs against the default branch; run it before merging anything that handles user input or SQL.

SUMMARY

Five working practices

One task, one context	`/clear` between stages. SPEC.md, PLAN.md and CLAUDE.md carry the state; the conversation does not.
Interview, do not dictate	Both documents were produced by answering the model's questions. The developer stayed the source of truth.
Thin vertical slices	Small enough to review, large enough to test end to end. One slice, one commit.
Verification is part of the slice	The skill does not tick a box without a passing suite; the reviewer does not approve without reading the diff; the human runs the application.
Permissions are the safety net	Allow the routine, ask for the irreversible, deny the secret. Never bypass them to go faster.

WHAT FOLLOWS

From hand-written helpers to product-grade tooling

Built in this module

One skill, one hook and one subagent, written by hand for one repository. They encode a procedure, display progress and review a commit.

Next module

Skills that generate the schema and evaluation files for *any* database from its URL · evaluations that test whether a skill triggers when it should · hooks that enforce rather than inform · packaging as a plugin for a team · measuring the context cost of all of it.

CARRY FORWARD The repository, the two YAML formats, and the evaluation pass rate recorded today.

REFERENCES

Documentation

Memory (CLAUDE.md)	code.claude.com/docs/en/memory
Permissions and modes	code.claude.com/docs/en/permissions · /permission-modes · /settings
Hooks	code.claude.com/docs/en/hooks
Skills	code.claude.com/docs/en/skills
Subagents	code.claude.com/docs/en/sub-agents
Commands and CLI	code.claude.com/docs/en/commands · /cli-reference · /code-review
Tools (AskUserQuestion)	code.claude.com/docs/en/tools-reference
Best practices · prompt library	code.claude.com/docs/en/best-practices · /prompt-library
Context window · worktrees	code.claude.com/docs/en/context-window · /worktrees · /sessions
Advisor · ~/.claude directory	code.claude.com/docs/en/advisor · /claude-directory
Effort · status line	platform.claude.com/docs/en/build-with-claude/effort · code.claude.com/docs/en/statusline

Manaranjan Pradhan · www.manaranjanp.com